

TBOT GATE REDESIGN — COMPLETE IMPLEMENTATION PLAN

Live day one, same discipline as the brain rebuild. Tier 1 → 4, dependency-ordered.

READ THIS FIRST, STEPHEN

- **Same day-one-live decision as the brain rebuild, same reasoning.** Everything here ships live today, not staged behind a pre-launch test. Gate Tier 3 specifically (the two Livermore-coupled items) carries the same real risk Tier 3's judges did — it gets the identical safety net: old and new computed side by side, every divergence logged with a real synthetic outcome, and a single flag that reverts it instantly if something looks wrong. No redeploy needed to pull back.
- **Order matters.** Tier 1 has zero dependencies — ship it first, ship it separately if useful. Tier 2 is independent of Tier 1. Tier 3 needs nothing from 1

or 2 but is the one piece carrying real risk. Tier 4 depends on Tier 3 already being live – it reuses machinery Tier 3 builds.

- **Business decisions, confirmed, not defaults:**

Conviction Margin is ON, starting value derived below from real log data, not invented. NY-open stays a fixed clock window – no change, confirmed as-is.

- Every fix below was checked against the real repo before being written down.

GATE TIER 1 – safe now, zero Livermore coupling, ships independently

1.1 – Remove sniper-filter from the live path, archive the logic

Why: confirmed dead – computes real analysis, result discarded. Removing the call cuts wasted computation; archiving the logic (not deleting it) keeps the displacement/coiled-spring detection available if it's ever given a real job.

Where: `council_aggregator.py`, wherever `_check_sniper_filter` is currently called.

```

# Remove this call entirely from the live
decision path:
# sniper_passed, sniper_details =
self._check_sniper_filter(df, signal)
# if not sniper_passed:
#     logger.debug(f"[DISPLACEMENT] Low
momentum candle – informational only, no
penalty")

# The function itself stays in the file,
untouched, with this added to its
docstring:
"""
[ARCHIVED – not called in the live path as
of this rebuild. The displacement/
coiled-spring/Turtle-breakout logic here is
real and well-built; it was
formally retired ("MRS Phase 0") with the
veto removed but the computation
never actually stopped running. Candidate
for reintroduction as a
Corroboration-layer evidence source, added
to JUDGE_SOURCE_REGISTRY if it
ever gets wired back in – do not silently
re-enable without that step.
"""

```

1.2 – STEP 8 (council branch): redundant check becomes a loud tripwire

Why: confirmed this re-checks values council's own internal logic already compared one step earlier –

mathematically redundant today. Converting it from silent INFO to loud ERROR turns dead code into a genuine safety net for the day something upstream desyncs `merged_details` from what council actually decided.

Where: `main.py`, STEP 8, council branch.

```
# Find:
if selected_mode == "council":
    min_score =
merged_details.get("required_score", 3.5)
    actual_score =
merged_details.get("total_score", 0)
    if signal != 0 and actual_score <
min_score:
    logger.info(
        f"[COUNCIL] Signal filtered:
{actual_score:.2f} < {min_score:.2f}"
    )
    signal = 0
    merged_details["reasoning"] = (
        f"Council score too low
({actual_score:.2f}/{min_score:.2f})"
    )

# Replace with:
if selected_mode == "council":
    min_score =
merged_details.get("required_score", 3.5)
    actual_score =
merged_details.get("total_score", 0)
    if signal != 0 and actual_score <
```

```

min_score:
    # This check is mathematically
    redundant with council's own internal
    # threshold comparison (confirmed
    2026-07-10) – by the time signal
    # reaches here, council already
    zeroed it if this were true using the
    # SAME numbers. If this ever fires,
    something upstream desynced
    # merged_details from what council
    actually decided. Loud on purpose.
    logger.error(
        f"[STEP8 TRIPWIRE]
{asset_name}: council said signal={signal}
but "
        f"STEP 8's re-check found
{actual_score:.2f} < {min_score:.2f} – "
        f"this should be mathematically
impossible. Investigate merged_details "
        f"drift between council's
return and this point."
    )
    signal = 0
    merged_details["reasoning"] = (
        f"STEP8_TRIPWIRE_FIRED
({actual_score:.2f}/{min_score:.2f})"
    )

```

1.3 – Economics gate reads real friction data, not a static guess

Why: confirmed the gate's actual formula (`risk` , `expected_reward` , `atr_fast` , a fixed 1.2:1 ratio)

never references cost at all – while C1 already built real, continuously-corrected per-asset friction data (`FRICTION_PENALTIES`) that never reaches this function. The gate whose entire job is "is this worth it after costs" currently checks "is this worth it before any cost."

Where: first, thread `shadow_trader` into `council_aggregator.py`'s constructor – same injection pattern as `outcome_pipeline`.

```
# CouncilAggregator.__init__ – add
alongside the existing injections:
def __init__(
    self, ..., mtf_integration=None,
    performance_tracker=None,
    outcome_pipeline=None,
    shadow_trader=None,
    use_macro_governor=True,
    use_gatekeeper=True,
):
    ...
    self.shadow_trader = shadow_trader #
for real friction data in the economics
gate
```

```
# main.py – all four
InstitutionalCouncilAggregator(...) call
sites, same as
# outcome_pipeline's four injection points:
InstitutionalCouncilAggregator(
```

```

    ...,
    outcome_pipeline=self.outcome_pipeline,
    shadow_trader=self.shadow_trader,
)

```

```

# _check_profit_economics_adaptive – add
real friction cost to the check:
def _check_profit_economics_adaptive(self,
    entry, stop_loss, first_tp_mult, atr_fast,
    asset_name=None):
    try:
        risk = abs(entry - stop_loss)
        if risk <= 0:
            return True

        expected_reward = risk *
first_tp_mult
        min_required = 0.5 * atr_fast

        # Real, observed friction cost –
replaces the implicit "cost is zero"
        # assumption. Falls back gracefully
if shadow_trader isn't injected
        # yet or has no data for this
asset.
        _friction_cost = 0.0
        if self.shadow_trader is not None
and asset_name:
            _friction_pct =
getattr(self.shadow_trader,
"FRICTION_PENALTIES", {}).get(
                asset_name.upper(), None
            )

```

```

        if _friction_pct is not None:
            _friction_cost =
(_friction_pct / 100.0) * entry

            expected_reward -= _friction_cost

            if expected_reward < min_required:
                logger.info(
                    f"[PROFIT GATE] Blocked -
Low Reward after real friction "
                    f"(reward
{expected_reward:.4f} < {min_required:.4f},
"
                    f"friction_cost=
{_friction_cost:.4f})"
                )
                return False

            return (expected_reward / risk) >=
1.2

    except Exception as e:
        logger.error(f"[PROFIT] Error:
{e}")
        return True

```

1.4 – Performance track record routes into outcome_pipeline

Why: confirmed zero existing connection between
performance-tracking data and the shared outcome

pipeline. Purely additive — feeds real data in, changes no live behavior.

Where: wherever the Performance Circuit Breaker / Dynamic Weighting logic reads win-rate/profit-factor data.

```
# Alongside the existing circuit-breaker
check, tag the read into the shared
pipeline:
if hasattr(self, "outcome_pipeline") and
self.outcome_pipeline is not None:
    self.outcome_pipeline.tag(
        asset=asset_name,
        direction="circuit_breaker_check",
        price=current_price,
        timestamp=datetime.now(timezone.utc),
        event_type=f"perf_track_wr=
{win_rate:.2f}_pf={profit_factor:.2f}",
    )
```

1.5 — Momentum-alignment joins the correlation registry

Why: confirmed genuinely consumed, genuinely independent evidence — but invisible to 5.1's correlation math since it isn't in

`JUDGE_SOURCE_REGISTRY`.

Where: `council_aggregator.py`, the registry itself (Brain Rebuild Part 0.7).

```

JUDGE_SOURCE_REGISTRY = {
    "trend":      [("independent", 0.90),
("livermore_1h", 0.10)],
    "momentum":  [("independent", 0.85),
("livermore_1h", 0.15)],
    "pattern":   [("independent", 1.0)],
    "structure": [("independent", 0.85),
("livermore_1h", 0.15)],
    "volume":    [("independent", 0.85),
("livermore_1h", 0.15)],
    "reversion": [("independent", 1.0)],
    "recent_momentum_alignment":
[("independent", 1.0)], # Gate Tier 1.5 –
real, dedicated candle-displacement
evidence, now visible to 5.1
}

```

1.6 – Conviction Margin: ON, with a real starting value

Why: confirmed complementary to 5.2, not redundant – catches a barely-separated race even when the losing side never cleared its own bar. Derived the starting value from real numbers already seen this session, not invented: GOLD's genuinely decisive SELL had a ~2.03 margin (buy=1.32, sell=3.35); the code's own illustrative "should be caught" example is a 0.1 margin (3.1 vs 3.0). A starting value needs to sit clearly below a real decisive case and clearly catch a near-tie.

Where: config.

```
"council_min_score_margin": 0.35
```

Layman: 0.35 comfortably lets GOLD's real ~2.0-margin decision through untouched, while catching anything close to the "3.1 vs 3.0" coin-flip shape the mechanism was built for. A real starting point, not a guess — revisit once more real margin data accumulates.

GATE TIER 2 — real evidence, wrong shape, lightweight validation before shipping

2.1 — Governor-filter: graduated, not binary

Why: genuinely dedicated daily-timeframe evidence, currently an absolute veto. Same fix shape as CHASE_HARD → CHASE-1.

Where: `_check_governor_filter` 's call site — instead of a hard pass/fail, return a required-score adjustment.

```
# Instead of gov_passed being a hard gate:  
gov_passed, trade_type, gov_confidence =  
self._check_governor_filter(...)
```

```

if not gov_passed:
    # No longer a hard block – scales the
    required_bar instead, same
    # pattern as _check_lifecycle_phase
    already uses successfully.
    _gov_penalty = 0.5 * (1.0 -
gov_confidence) # weaker daily-trend
disagreement = smaller bump
    required_score += _gov_penalty
    logger.info(
        f"[GOVERNOR] {asset_name}: daily
trend disagrees (conf={gov_confidence:.2f})
"
        f"- required_score raised by
{gov_penalty:.2f}, not blocked outright"
    )

```

Needs before shipping: a quick check against a real log – confirm the confidence values this actually produces make sense scaled this way, same lightweight validation ATR Wick Trap gets below. Not full shadow mode – this doesn't touch Livermore.

2.2 – ATR Wick Trap: graduated, not binary

Where: `validate_candle_structure`'s call site.

```

_wick_valid = validate_candle_structure(df,
self.asset_type, direction=...,
regime_confidence=regime_conf,
regime_aligned=_trap_regime_aligned)
if not _wick_valid:

```

```
# Graduated instead of absolute – a  
wick trap firing once costs a real  
# but recoverable penalty; the  
threshold itself already scales 1.0x-1.5x  
# ATR by regime, this just stops the  
FINAL step from being all-or-nothing.  
required_score += 0.4  
logger.info(f"[TRAP] {asset_name}: wick  
structure flagged – required_score +0.4,  
not blocked outright")
```

Needs before shipping: same lightweight real-log
check as 2.1.

GATE TIER 3 – Livermore-coupled, live-decision-affecting, full shadow- mode discipline

3.1 – Counter-trend block: graduated, computed alongside the old hard veto

Why: structurally undermines Tier 3's whole premise
– vetoes a signal TREND earned independently, solely
because Livermore's 1H state disagrees. Same
shadow-mode treatment
TREND/MOMENTUM/PATTERN got, because this is
exactly as consequential a change to how Livermore
affects a live decision.

Where: `main.py`, replacing the POST-SIGNAL
LIVERMORE COUNTER-TREND BLOCK.

```
_gate3_shadow_mode =
self.config.get("phase_config",
{}).get("gate3_shadow_enabled", False) #
False = new graduated behavior live from
day one, same posture as Tier 3's judges

if signal != 0:
    _lsm_post =
details.get("livermore_state_1h")
    if _lsm_post is None:
        _cs_post =
mtf_regime.get("composite_state")
        if _cs_post is not None:
            _lsm_post = getattr(_cs_post,
"livermore_state_1h", None)

    _old_would_block = (
        (_lsm_post == "NATURAL_RETRACEMENT"
and signal < 0)
        or (_lsm_post == "NATURAL_REBOUND"
and signal > 0)
    )

    if _old_would_block:
        # New behavior: graduated, scaled
by retracement depth via D4's
        # companion piece (3.2) – how far
price already sits from the
        # retracement's own recent extreme,
in ATRs. Deeper retracement
```

```

        # already run = smaller bump (more
likely genuinely reversing);
        # shallow, fresh retracement =
larger bump (more likely to resume
        # the original trend, which is what
this block exists to protect).
        _retr_depth_atr =
self._estimate_retracement_depth(df,
_lsm_post) # new helper, 3.2
        _new_bump = max(0.15, 0.6 - (0.15 *
_retr_depth_atr))
        _new_would_block = False #
graduated version never hard-blocks – it
raises the bar instead
        required_score += _new_bump

        _old_signal_for_log = signal if not
_gate3_shadow_mode else signal
        if _gate3_shadow_mode:
            # Emergency brake: revert to
the exact old hard-veto behavior.
            signal = 0
            details["reasoning"] =
"livermore_counter_trend_block"

self._shadow_open_blocked(asset_name,
_old_signal_for_log, details, df,
current_price, "gate3_counter_trend_old",
asset_cfg)
        else:
            logger.info(
                f"[COUNTER-TREND]
{asset_name}: {_lsm_post} counter-trend, "
                f"graduated (+

```

```

    {_new_bump:.2f} required_score) not blocked
    - "
        f"depth=
    {_retr_depth_atr:.2f}ATR"
    )
    # Log the divergence – what the
    OLD hard veto would have done,
    # for the same real comparison
    Tier 3's judges got.
    details["gate3_divergence"] = {
        "old_would_block": True,
    "new_bump_applied": _new_bump,
        "gate_label":
    f"gate3_counter_trend_new_{asset_name}",
    }

```

```

# Wherever details gets consumed downstream
(same pattern as Tier 3's
# divergence tagging, X.2-style):
if details.get("gate3_divergence"):
    _div = details["gate3_divergence"]
    self._shadow_open_blocked(asset_name,
    signal, details, df, current_price,
    _div["gate_label"], asset_cfg)

```

Emergency brake: flip `gate3_shadow_enabled` to `True`, instantly reverts to the exact old hard-veto behavior, no redeploy – identical mechanism to `tier3_shadow_enabled`.

3.2 — D4's new transition type: does the retracement hold or fail

Why: the actual live monitor this redesign needs — D4's existing watch only covers MAIN-to-opposite-MAIN, a different and larger risk than a retracement failing.

Where: `veteran_trade_manager.py`, a genuinely new method, kept independently toggleable from 3.1 per the earlier debate — so a shadow-mode divergence can be attributed to one piece or the other, not bundled.

```
def _estimate_retracement_depth(self, df,
    lsm_state: str) -> float:
    """New helper (Gate Tier 3.2) — how far
    price has already pulled back
    from its own recent extreme, in ATRs.
    Feeds 3.1's graduated bump at
    entry; also the live monitor once a
    counter-trend-adjacent trade is open."""
    try:
        atr = self._calculate_atr()
        if atr <= 0:
            return 1.0 # neutral default,
            doesn't over- or under-penalize
        lookback = df.tail(20)
        if lsm_state ==
"NATURAL_RETRACEMENT":
            _extreme =
lookback["high"].max()
```

```

        _current = df["close"].iloc[-1]
        return max(0.0, (_extreme -
_current) / atr)
        elif lsm_state ==
"NATURAL_REBOUND":
            _extreme =
lookback["low"].min()
            _current = df["close"].iloc[-1]
            return max(0.0, (_current -
_extreme) / atr)
        return 1.0
    except Exception:
        return 1.0

def _check_retracement_holding(self) ->
Optional[str]:
    """Called from
update_with_current_price alongside the
existing
    MAIN-to-opposite-MAIN watch – a
genuinely different transition type,
    not an extension of it. Returns a
signal if the retracement this
    trade was opened against is now failing
(resuming the original
    trend) rather than holding."""
    if self.livermore_state_1h not in
("NATURAL_RETRACEMENT", "NATURAL_REBOUND"):
        return None
    _current_depth =
self._estimate_retracement_depth(self._live
_df, self.livermore_state_1h)
    _entry_depth = getattr(self,
"_retracement_depth_at_entry", None)

```

```

        if _entry_depth is None:
            self._retracement_depth_at_entry =
            _current_depth
            return None
        if _current_depth < _entry_depth * 0.5:
            return "retracement_failing" #
already recovering more than half its own
depth – original trend likely resuming
            return None

```

```

# In on_new_bar/update_with_current_price,
alongside the existing MAIN-flip watch:
_retr_signal =
self._check_retracement_holding()
if _retr_signal == "retracement_failing":
    self.runner_trail_atr_multiplier =
_config_trail * _sec_trail * 0.5 #
tighten, don't lock breakeven outright –
genuinely different confidence than a
confirmed MAIN flip

```

Emergency brake: independent of 3.1's flag – this can be disabled on its own

(`gate3_d4_extension_enabled` , default `True`)

without reverting the entry-side gate change, per the attribution requirement from the earlier debate.

3.3 – `lsm_regime_disagreement_gate_enabled` :
lighter shadow scope

Why: same category, genuinely simpler mechanism — skip resolution vs. a coupled two-piece redesign. Doesn't need the same weight of infrastructure.

```
_gate_disagree_shadow =
self.config.get("phase_config",
{}).get("disagreement_gate_shadow_enabled",
False)
if _regime_lsm_disagree:
    if _gate_disagree_shadow:
        logger.info(f"[COUNCIL]
{self.asset_type} disagreement gate: OLD
behavior (skip) — shadow mode")
        # existing skip behavior, unchanged
    else:
        # New: graduated required_score
raise instead of a full skip
        required_score += 0.5
        logger.info(f"[COUNCIL]
{self.asset_type} disagreement —
required_score +0.5, not skipped")
```

Emergency brake:

`disagreement_gate_shadow_enabled` → `True` .

**GATE TIER 4 — deep integration,
depends on Tier 3 being live**

4.1 — 4.5's alert layer reuses lifecycle-phase's real classification

Why: `_check_lifecycle_phase` already does a richer exhaustion read (ADX decline + overextension + RSI divergence) than 4.5 currently rebuilds from raw flags. Reuse the real tool continuously, not just at entry.

```
# In _check_alert_conditions, alongside the
existing checks:
if self._live_cs is not None and
self._live_df is not None:
    _, _phase_label =
self.council_ref._check_lifecycle_phase(
    self._live_df, 1 if self.side ==
"long" else -1,
    self._live_cs.get("adx", 0) if
isinstance(self._live_cs, dict) else
getattr(self._live_cs, "adx", 0),
    0, # required_score unused for
this read-only call
    governor_data={"composite_state":
self._live_cs},
    )
    if _phase_label == "EXHAUSTED":

signals_fired.append("lifecycle_phase:
EXHAUSTED (reused entry-time classifier)")
```

Needs: VTM needs a `council_ref` reference to call this — same injection-threading pattern as

`outcome_pipeline / telegram / aggregators`, one more addition to the same chain.

4.2 – D4's retracement outcomes feed

`outcome_pipeline`

Why: closes the loop 3.2 opens – the entry-time bump (3.1) is currently a reasoned estimate; this is what eventually replaces it with a real, checked number.

```
# Wherever a position closes, alongside the
existing D2/D3 tracking:
if getattr(_vtm,
    "_retracement_depth_at_entry", None) is not
None:
    _outcome = "held" if reason not in
    (ExitReason.STOP_LOSS.value,) else "failed"
    self.outcome_pipeline.tag(
        asset=_vtm.asset,
        direction=f"retracement_{_outcome}",
        price=_vtm.entry_price,
        timestamp=datetime.now(timezone.utc),

        event_type=f"gate3_retracement_depth=
        {_vtm._retracement_depth_at_entry:.2f}atr",
    )
```

4.3 – Zone ladder's rejection leg gets a displacement floor

Why: momentum-alignment's design principle (real net movement, not just a technical touch) borrowed as its own small addition — not a merger of the two mechanisms, confirmed they measure different things.

```
# In the zone ladder's confirmation logic,
# alongside the existing
# proximity-then-move-away rejection check:
def _is_genuine_rejection(self, df, level,
    atr, direction, n_candles=2):
    """Borrowed principle from momentum-
    alignment (Gate Tier 1.5) — a
    rejection only counts if price actually
    traveled away from the level,
    not just closed on the technically-
    correct side of it."""
    if atr <= 0 or len(df) < n_candles + 1:
        return True # insufficient data,
        don't block on this alone
    _recent_closes = df["close"].iloc[-
    (n_candles):].values
    _displacement = abs(_recent_closes[-1]
    - level)
    return _displacement >= (0.3 * atr) #
    same floor concept, smaller multiple given
    this is a rejection not an adverse-momentum
    check
```

Confirmed, no code needed

NY-open — stays a fixed clock window, as-is. No change.

That's the complete plan, four tiers, everything live today except the two shadow-mode flags in Tier 3, which are set to fire the new graduated behavior by default with the old behavior one flag away.